

Data Analysis for Experimental Physics with Machine Learning

A.A. 2025/26, Alessandro Foglizzo

Appunti non esaustivi del corso di Data Analysis for Experimental Physics with Machine Learning, tenuto dal prof. Paolo Meridiani presso il dipartimento di Fisica dell'Università di Torino nell'anno 2025/2026.

Concetti fondamentali

- Dataset: insieme di coppie (x_i, y_i) , per $i=1\dots N$ eventi
- Features: insieme di variabili (indice $j=1\dots m$ features) che caratterizzano un evento i : $x_i = (x_{i1}, \dots, x_{ij}, \dots, x_{im})$
- Label: etichetta associata all'evento, è la variabile che lo classifica e viene utilizzata per il training, è il risultato atteso
- Modello: funzione matematica che, prendendo in pasto le features, attraverso i parametri θ ottenuti durante il training, ottiene la predizione su $\hat{y}$
- Funzione oggettiva: $\text{obj}(\theta) = L(\theta) + \Omega(\theta)$, misura il grado di accordo tra il fit e i dati. es MSE
- Loss function $L(\theta)$: misura quanto il modello è predittivo rispetto al training set $L(\theta) = \sum_i (y_i - \hat{y}_i)^2$
- Regularization term $\Omega(\theta)$: controlla la complessità del modello e aiuta ad evitare l'overfitting
- variance-bias tradeoff: equilibrio tra un modello predittivo e semplice
- Training error: $E_{out} = Bias^2 + Var + Noise$
- Varianza: misura quanto cambia la predizione del modello cambiando il dataset di apprendimento, e quindi la sensibilità del modello alle fluttuazioni statistiche del dataset
- Bias: o distorsione, indica la differenza tra le assunzioni sulla semplicità del modello e la complessità dei dati reali, ed è la differenza tra il valore predetto e quello reale
- Overfitting: alta varianza e basso bias, dato da modelli troppo complessi. Errore di training molto basso, errore di test molto alto (non generalizza)
- Underfitting: alto bias e bassa varianza, dato da modelli troppo semplici. Errore di training e di test molto alti.
- Batch: porzione/sottoinsieme del dataset di training
- Iperparametro: configurazione esterna al modello, che non viene appresa ma va fornita prima dell'addestramento
- Epoch: singola iterazione completa su tutto il dataset di addestramento
- Parametri di nuisance: parametri di disturbo, che rappresentano le incertezze sistematiche che possono cambiare la forma o la normalizzazione della pdf. Possono essere di natura teorica o sperimentale

1 Neural Networks

Estrazione automatica delle features per trovare quelle migliori per un dato task.

Ogni neurone, sommando i pesi associati a ciascuna feature, misura una distanza da un iperpiano che viene non-linearizzata per esaltarne le differenze (4.5).

Diversi tipi di non linearità: Perceptron (θ), sigmoide, tanh, ReLU, Leaky ReLU, ELU (4.6).

L'input dei neuroni del layer denso n ($z = W^T x$) è dato dall'output (attivazioni, $a = \sigma(z)$) dei neuroni del layer $n - 1$, ed ha pesi indipendenti. Tutti i layer, meno l'input e l'output, sono nascosti (hidden): **Feed Forward Fully Connected Neural Network**. L'output dipende dal task:

- Classificazione binaria: output singolo, con attivazione a sigmoide
- Regressione lineare 1D: singolo neurone con attivazione lineare
- Multiclassification: K neuroni con attivazione softmax

Senza le non linearità, i vari strati sarebbero semplificabili a una mera moltiplicazione di matrici, che dà una matrice (lineare).

Universal Approximation Theorem: data una funzione $y = f(x)$ e un $\varepsilon > 0$, esiste un deep network $y = f_w(x)$ di profondità o larghezza arbitraria per cui

$$\sup ||f(x) - f_w(x)|| < \varepsilon$$

Quindi, una NN sufficientemente larga o profonda può approssimare una qualsiasi funzione continua su un dominio compatto. Questo però non garantisce che sia *learnable*.

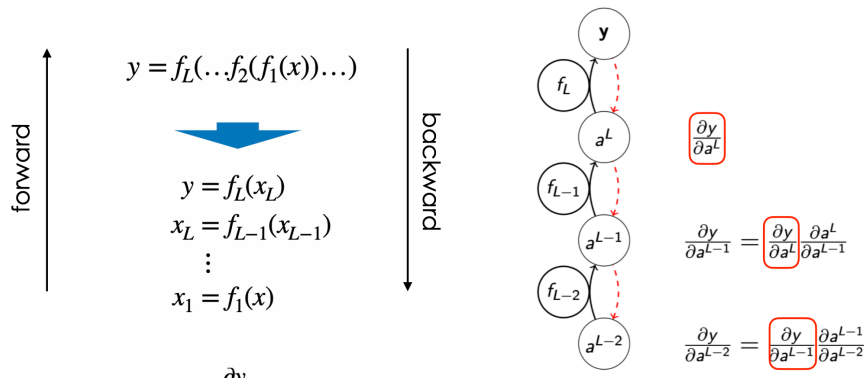
Training NN: a definita la LOSS FUNCTION in base al task e alle assunzioni: **cross-entropy** per classificazione, **MSE** per regressione. Dopodiché, trovare i pesi $\hat{w}$ che massimizzano la massima verosimiglianza $L(w)$. Per il *gradient descent*: $\frac{\partial L(w)}{\partial w}$.

Backward Propagation Algorithm: è l'algoritmo per calcolare i gradienti delle loss function. Sappiamo che che

$$y = f_L(\dots f_2(f_1(x))\dots)$$

con

$$y = f_L(x_L) \text{ e } x_L = f_{L-1}(x_{L-1})$$

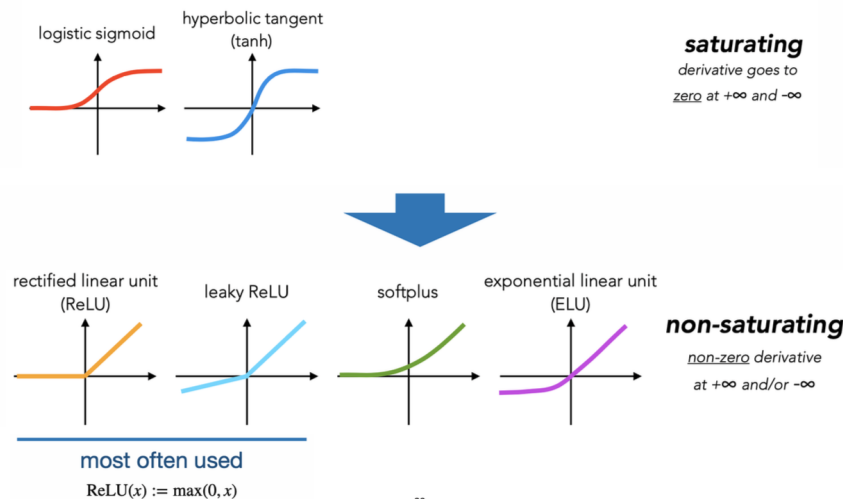


Per calcolare $\frac{\partial y}{\partial x_l}$ con $l \in 1, \dots, L$ dobbiamo applicare la chain rule. Partendo quindi dalla derivata all'output $y = f_L$, calcoliamo a ritroso tutte le $\frac{\partial y}{\partial x_l}$. (4.14)

La backpropagation è fatta in automatico dalla maggior parte dei pacchetti di deep learning. L'unica condizione per poter definire un tipo di layer è saperne definire il gradiente.

Le non linearità *saturating* hanno la derivata molto piccola praticamente ovunque, quindi mandano il gradiente a zero. Inoltre la sigmoide è problematica perché non è centrata in zero e contiene un e^x , lento computazionalmente.

Al contrario, la ReLU è molto usata e fornisce rapida convergenza e computazione; non è però centrata in zero (possibile instabilità, vedere batch normalization).

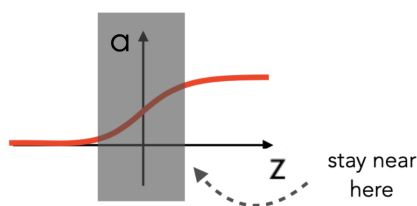


L'obiettivo dell'**inizializzazioni dei pesi** è mantenere attivazioni stabili lungo i vari layer. Vorremmo che $\text{Var}(a) \approx \text{Var}(z)$ per evitare che i segnali esplodano o svaniscano (vanishing gradients) durante la propagazione. Notazione:

- W : matrice dei pesi
- $\mathcal{N}(\mu, \sigma^2)$: distribuzione normale con media μ e varianza σ^2
- $\sim$: estratto dalla distribuzione
- $n_{in/out}$: numero di neuroni in ingresso/uscita del layer (fan in/out)

Esistono diverse inizializzazioni:

- **Zero Initialization**: mette tutti i pesi a zero, da evitare per problemi di simmetria che impediscono l'apprendimento
- **Lecun Initialization**: $W \sim \mathcal{N}(0, \frac{1}{n_{in}})$
- **He (Kaiming) Initialization**: progettata per ReLU e Leaky ReLU, $W \sim \mathcal{N}(0, \frac{2}{n_{in}})$ (il fattore 2 compensa l'azzeramento, in media, della metà delle attivazioni)
- **Xavier (Glorot) Initialization**: progettata per sigmoide e tanh, $W \sim \mathcal{N}(0, \frac{1}{n_{in} + n_{out}})$ (cerca un bilanciamento tra il passo avanti e quello indietro)



Batch Normalization: trasformazione che riporta gli input delle attivazioni ad avere media 0 e varianza 1 e quindi rimanere all'interno del range dinamico della funzione non lineare. Si calcola la media μ_B e la varianza σ_B^2 delle attivazioni all'interno di ogni batch, dopodiché si introducono due parametri apprendibili: una scala γ e uno spostamento β .

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

La Batch Normalization sembra funzionare perché rende la loss function più smooth nei parametri.

NB: utilizzando la ReLU, è comune applicare la BN **dopo** l'attivazione, perché la ReLU rimuoverebbe i valori negativi.

Connettività Residuale: rispetto a una rete con connettività sequenziale, in cui l'informazione deve passare attraverso tutti i layer per arrivare all'output e ogni layer trasforma completamente l'input ricevuto, questa permette di trovare dei percorsi alternativi più brevi. Invece di imparare una funzione complessa $H(x)$, si parte da un input x e si impara solo la funzione differenza tra $H(x)$ e x . Matematicamente, $H(x) = F(x) + x$. Se x è già una buona approssimazione di $H(x)$, la rete dovrà solo imparare una piccola correzione $F(x)$. In questo modo, il termine $+x$ evita che i gradienti vadano a zero e permette una backpropagation più fluida; inoltre, l'informazione è preservata anche quando $F(x)$ è molto piccola.

Learning Rate α : si dovrebbe iniziare l'apprendimento con un learning rate alto, in modo da convergere più rapidamente ed evitare minimi molto pronunciati. Dopodiché, per convergere a un risultato va diminuito il learning rate, ma non troppo presto e in più fasi in modo da evitare di finire in un minimo locale. Si possono adottare diverse strategie:

- Exponential Decay: decadimento graduale nel tempo, $\alpha_t = \alpha_0 \exp^{-\lambda t}$
- Cosine Annealing: riduzione graduale con una funzione coseno, $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(\frac{t}{T}\pi))$
- 1Cycle Policy: aumenta il learning rate all'inizio, poi decade per il fine tuning

Riduzione dell'Overfitting (Regolarizzazione): tecniche di regolarizzazione per ridurre l'errore di generalizzazione limitando la complessità del modello e restringendo lo spazio dei parametri.

- Early Stopping: interrompe l'addestramento se la rete non è migliorata per n epochs. Iperparametro: *patience*.
- Dropout: impostazione di alcuni neuroni a zero, scelti randomicamente con probabilità p (iperparametro). I pesi per la predizione vanno riscalati di un fattore p , ma è fatto in automatico dalle librerie di ML: $w_{test}(p) = p w_{train}(p)$

Consiglio: monitorare sempre la training history, controllando le figure di merito (loss, accuracy, MSE) per i dataset training e validazione. Inoltre, evitare numeri molto piccoli e molto grandi e controllare se ci sono NaN quando il modello non migliora.

2 Calibrazione della risposta di un rivelatore

Con **risposta di un rivelatore** ci si riferisce alla misura ottenuta da un rivelatore che ha ricevuto uno specifico input. I rivelatori non producono misure esatte e per uno stesso input fisico y l'output x potrebbe variare per effetti stocastici (noise e risoluzione) o sistematici (soglie di misurazione,...). Quindi, la risposta x del rivelatore è governata da una legge di *probabilità condizionale* $p(x|y)$.

La calibrazione del detector si può formalizzare come il problema inverso, $p(y|x)$.

Features in input: osservabili x del rivelatore (energia ricostruita, posizione, forma degli sciame,...)

Target: quantità y (energia true o θ_{true} , normalmente E_{true}/E_{reco} o $\theta_{true} - \theta_{reco}$).

Questa ricostruzione subisce tutti gli effetti del detector: noise, risoluzione, bias, non linearità. Parametrizzare $p(y|x)$ è spesso complesso, il goal è apprendere questa funzione per caratterizzare il comportamento del detector e correggere i bias.

Una grossa sfida è data dalle distribuzioni della risposta: non gaussiane, asimmetriche e con lunghe code. Quindi la media potrebbe non corrispondere al valore più probabile.

b-jet: gli adroni con b decadono quasi sempre in adroni c , in parte con canali leptonici, e quindi aggiungono particelle leggere (neutrini, muoni, u/d-jets) che finiscono "fuori dal cono". Per questo, è necessaria una calibrazione specifica. per i b-jet

3 Classificatori per analisi di eventi HEP

Bosone di Higgs: $m_H = 125$ GeV, $\tau_H \sim 10^{-25}$ s, scalare.

Produzione: accoppiamento con fermioni (in particolare t che ha la massa, e quindi l'accoppiamento con l'Higgs, maggiore) ggH (gluon fusion con loop t 90%) e ttH ($2g \rightarrow t\bar{t}H$); accoppiamento con bosoni vettori VBF (vector boson fusion, $2q \rightarrow qqWW/qqZZ$), VH ($q\bar{q} \rightarrow W/Z \rightarrow W/Z+H$).

Decadimento nelle **Higgs Big Five**:

Channel	$\sigma(m_H)/m_H$ [%]	m_H range [GeV]	S/B (typ.)
$H \rightarrow ZZ \rightarrow 4\ell$	1–2	110–1000	High (~ 1)
$H \rightarrow \gamma\gamma$	1	110–150	Moderate ($\sim 0.05\text{--}0.2$)
$H \rightarrow WW \rightarrow 2\ell 2\nu$	20	110–660	Low (~ 0.01)
$H \rightarrow \tau\tau$	15	110–145	Low (~ 0.02)
$H \rightarrow b\bar{b}$	10	110–135	Very low ($\sim 0.001\text{--}0.01$)

I canali con migliore risoluzione di massa e più grande S/B sono i più adatti per scoprire e misurare il bosone di Higgs.

Flow di un'analisi in HEP

1. Definire che cosa si vuole misurare

2. Scegliere uno stato finale (o canale)
3. Identificare i processi che compongono il fondo
4. Definire come selezionare gli eventi
5. Definire un'osservabile (numero di eventi, massa invariante, asimmetrie,...) con istogrammi
6. Ottenere le misure e le loro incertezze

Analisi $H \rightarrow \gamma\gamma$

Ricerca di un picco piccolo e stretto su un grande fondo che decade esponenzialmente. È necessaria una buona risoluzione energetica e spaziale del calorimetro EM per determinare l'angolo tra i due fotoni.

Bump-Hunt Analysis: il fondo che decade permette una stima robusta del fondo direttamente nella zona del segnale. In questo caso, su uno spettro di massa invariante $m_{\gamma\gamma}$.

$$m_{\gamma\gamma} \sqrt{2E_1 E_2 (1 - \cos \theta_{12})}$$

Simple-cut analysis: selezione degli eventi che contengono 2 fotoni isolati con $p_T > 30$ GeV e $|\eta| < 2.5$: definisce una regione di segnale usando tagi rettangolari sulle features osservabili (p_T, η , isolation).

Detta σ_m la risoluzione del picco della massa, il rapporto S/B è proporzionale a $1/\sigma_m$: migliorare la risoluzione della massa porta direttamente a una migliore separazione del segnale dal rumore. Questo si può fare con una regressione energetica (DT o NN) per correggere la risposta dell'ECAL, oppure con una stima di σ_m evento per evento, assegnando poi i pesi maggiori agli eventi con miglior risoluzione.

Un'altra via per massimizzare S/B è migliorare la selezione degli eventi, cioè ridurre il False Positive Rate (FPR) mantenendo lo stesso True Positive Rate (TPR).

- identificazione di fotoni isolati, per ridurre i fake-photons
- cinematica del sistema di-photon: usare la correlazione tra i due fotoni per separare $H \rightarrow \gamma\gamma$ dagli eventi di QCD
- evitare che l'output dei classificatori sia correlato alla massa invariante dei due fotoni, per mantenere il "bump-hunt"

Di-photon classifier: Boosted Decision Tree (BDT) addestrato su campioni simulati di segnali di Higgs (attraverso tutti i canali di produzione) e di fondo (mix di di-photons di QCD, γ +jet e di-jet fake).

Per evitare che il classificatore impari ad usare la massa per discriminare i segnali, dobbiamo dare in input features che siano indipendenti da $m_{\gamma\gamma}$.

1	$p_T^1/m_{\gamma\gamma}$	Mass independent kinematics
2	$p_T^2/m_{\gamma\gamma}$	
3	η_1	
4	η_2	
5	$\cos \Delta\phi_{\gamma\gamma}$	
6	$\sigma_m/m_{\gamma\gamma}$ (Right Vtx Hypothesis)	Mass resolution
7	$\sigma_m/m_{\gamma\gamma}$ (Wrong Vtx Hypothesis)	
8	p_{vtx}	
9	IDMVA ₁	Photon Identification score
10	IDMVA ₂	

Categorizzare gli eventi in base alla purezza del segnale migliora la sensibilità dell'intera analisi.

Forza del segnale: (6.16 per la funzione di likelihood con i parametri)